

Discesa del Gradiente Stocastica (SGD)

Metodi Statistici e Matematici per il Machine Learning

Traduzione italiana — materiale originale a cura di Davide Evangelista

Quando alleniamo un modello di Machine Learning, tipicamente abbiamo a che fare con un dataset

$$(X, Y) = \{(x^{(i)}, y^{(i)})\}_{i=1}^N,$$

e un **modello parametrico** $f_{\Theta}(x)$ i cui parametri $\Theta \in \mathbb{R}^d$ (chiamati anche **pesi**) devono essere appresi. La **fase di training** viene formulata come un **problema di ottimizzazione**, in cui cerchiamo parametri Θ tali che

$$f_{\Theta}(x^{(i)}) \approx y^{(i)} \quad \text{per ogni } i = 1, \dots, N.$$

Per quantificare questa approssimazione, introduciamo una **funzione di perdita** $\mathcal{L}(\Theta) := \mathcal{L}(\Theta; X, Y)$, che misura quanto le predizioni del modello si discostino dai target reali.

1 Minimizzazione del rischio empirico

Nella maggior parte dei problemi ML, la loss può essere espressa come una somma (o media) di **perdite campione per campione**:

$$\mathcal{L}(\Theta; X, Y) = \frac{1}{N} \sum_{i=1}^N \ell_i(\Theta; x^{(i)}, y^{(i)}),$$

dove ℓ_i quantifica l'errore per l' i -esimo campione di training.

Il problema di ottimizzazione del training diventa:

$$\Theta^* = \arg \min_{\Theta} \mathcal{L}(\Theta; X, Y) = \arg \min_{\Theta} \frac{1}{N} \sum_{i=1}^N \ell_i(\Theta; x^{(i)}, y^{(i)}).$$

Il metodo di **Discesa del Gradiente (GD)** (visto nella sezione precedente) aggiorna i parametri iterativamente come:

$$\begin{cases} \Theta^{(0)} \in \mathbb{R}^d, \\ \Theta^{(k+1)} = \Theta^{(k)} - \eta_k \nabla_{\Theta} \mathcal{L}(\Theta^{(k)}; X, Y) = \Theta^{(k)} - \eta_k \frac{1}{N} \sum_{i=1}^N \nabla_{\Theta} \ell_i(\Theta^{(k)}; x^{(i)}, y^{(i)}). \end{cases}$$

Qui $\eta_k > 0$ indica il **learning rate** (o passo di aggiornamento).

2 Esempio: Errore Quadratico Medio (MSE)

Una loss comune nei problemi di regressione è l'**Errore Quadratico Medio (MSE)**:

$$\mathcal{L}(\Theta; X, Y) = \frac{1}{N} \sum_{i=1}^N (f_{\Theta}(x^{(i)}) - y^{(i)})^2 = \sum_{i=1}^N \underbrace{\frac{1}{N} (f_{\Theta}(x^{(i)}) - y^{(i)})^2}_{=: \ell_i(\Theta; x^{(i)}, y^{(i)})}.$$

Per la regola della catena:

$$\nabla_{\Theta} \ell_i(\Theta; x^{(i)}, y^{(i)}) = \frac{2}{N} (f_{\Theta}(x^{(i)}) - y^{(i)}) \nabla_{\Theta} f_{\Theta}(x^{(i)}),$$

quindi:

$$\nabla_{\Theta} \mathcal{L}(\Theta; X, Y) = \frac{2}{N} \sum_{i=1}^N (f_{\Theta}(x^{(i)}) - y^{(i)}) \nabla_{\Theta} f_{\Theta}(x^{(i)}).$$

Pertanto, un'iterazione della Discesa del Gradiente diventa:

$$\Theta^{(k+1)} = \Theta^{(k)} - \eta_k \frac{2}{N} \sum_{i=1}^N (f_{\Theta}(x^{(i)}) - y^{(i)}) \nabla_{\Theta} f_{\Theta}(x^{(i)}).$$

3 Motivazione per la Discesa del Gradiente Stocastica

Sebbene il calcolo di ogni singolo $\nabla_{\Theta} \ell_i(\Theta; x^{(i)}, y^{(i)})$ sia poco costoso, il gradiente completo $\nabla_{\Theta} \mathcal{L}(\Theta; X, Y)$ richiede di sommare su tutti gli N campioni, il che è un costo proibitivo quando N è molto grande.

L'algoritmo di **Discesa del Gradiente Stocastica (SGD)** affronta questo problema sostituendo il **gradiente esatto** con un'**approssimazione stocastica** calcolata su un sottoinsieme casuale di dati (un *mini-batch*).

4 L'algoritmo SGD

Sia $N_{\text{batch}} \ll N$ la **dimensione del batch**, e sia $\mathcal{M}_k \subset \{1, \dots, N\}$ un sottoinsieme casuale di indici tale che $|\mathcal{M}_k| = N_{\text{batch}}$.

Ad ogni iterazione k :

1. **Campiona un mini-batch:** seleziona casualmente un sottoinsieme $\mathcal{M}_k$ (un **batch**) dal dataset (tipicamente senza reinserimento).
2. **Calcola il gradiente approssimato:**

$$\nabla_{\Theta} \mathcal{L}(\Theta^{(k)}; \mathcal{M}_k) = \frac{1}{N_{\text{batch}}} \sum_{i \in \mathcal{M}_k} \nabla_{\Theta} \ell_i(\Theta^{(k)}; x^{(i)}, y^{(i)}).$$

3. **Aggiorna i parametri:**

$$\Theta^{(k+1)} = \Theta^{(k)} - \eta_k \nabla_{\Theta} \mathcal{L}(\Theta^{(k)}; \mathcal{M}_k).$$

4. **Ripeti** finché tutti i dati non sono stati usati una volta. Quando l'intero dataset è stato processato, diciamo che è stata completata una **epoca** (*epoch*) di SGD. L'algoritmo viene tipicamente eseguito per un numero fisso E di epoche.

Nota

Figura del testo originale (non riprodotta qui): uno schema che illustra la suddivisione del dataset in mini-batch e il ciclo di campionamento/aggiornamento ad ogni iterazione, ripetuto per più epoche.

5 Confronto con la Discesa del Gradiente completa

Proprietà	Discesa del Gradiente (GD)	Discesa del Gradiente Stocastica (SGD)
Calcolo del gradiente	Intero dataset	Sottoinsieme casuale (mini-batch)
Costo per iterazione	Alto (dipende da N)	Basso (dipende da N_{batch})
Varianza	Deterministica	Stocastica
Convergenza	Regolare e stabile	Rumorosa, ma spesso più veloce
Scalabilità	Scarsa per dataset grandi	Ottima

La stocasticità introduce **rumore** negli aggiornamenti dei parametri, ma questo rumore può aiutare l'ottimizzatore a **sfuggire da minimi locali poco profondi** e spesso migliora la generalizzazione.

6 Implementazione tipica

```
import numpy as np

def SGD(f, grad_f, X, Y, Theta0, lr=1e-2, batch_size=32, epochs=10):
    """
    Implementazione semplificata della Discesa del Gradiente Stocastica (SGD).
    f: funzione di perdita, grad_f: gradiente rispetto a Theta
    X, Y: dataset
    Theta0: parametri iniziali
    lr: learning rate
    """
    Theta = Theta0.copy()
    N = len(X)
    for epoch in range(epochs):
        # Mescola gli indici dei dati
        idx = np.random.permutation(N)
        for start in range(0, N, batch_size):
            batch_idx = idx[start:start+batch_size]
            grad = grad_f(Theta, X[batch_idx], Y[batch_idx])
            Theta -= lr * grad
    return Theta
```

7 Osservazioni

- La **dimensione del batch** controlla il compromesso tra costo computazionale e accuratezza del gradiente. Batch piccoli introducono più rumore stocastico; batch grandi si avvicinano al comportamento della GD completa.
- Il **learning rate** η_k è solitamente costante oppure segue uno schema decrescente, ad es.

$$\eta_k = \frac{\eta_0}{1 + \gamma_k}, \quad \gamma > 0.$$

- In pratica, gli ottimizzatori moderni (ad es. Adam, RMSProp) estendono SGD con passi adattivi e momentum, che discuteremo più avanti nel corso.

Esercizio (Approssimazione mini-batch)

Per un vettore di ingresso $x^{(i)}$ di dimensione d , consideriamo la funzione

$$\mathcal{L}(\Theta; X, Y) = \frac{1}{N} \sum_{i=1}^N (\Theta^T x^{(i)} - y^{(i)})^2,$$

dove $\Theta \in \mathbb{R}^d$.

1. Per $d = 1$, definisci un dataset sintetico X, Y dove X è un insieme di $N = 100$ elementi equispaziati nell'intervallo $[0, 1]$, mentre $Y = 2X + 1 + e$, con $e \sim \mathcal{N}(0, 0.01)$.
2. Implementa la Discesa del Gradiente e la Discesa del Gradiente Stocastica per questa loss.
3. Confronta il numero di iterazioni e il costo computazionale.
4. Traccia l'evoluzione della loss lungo le epoche per diverse dimensioni del batch ($N_{\text{batch}} = 1, 10, N$).

Esercizio (Varianza del gradiente stocastico)

Per la stessa loss dell'esercizio precedente, calcola il gradiente su più batch casuali $\mathcal{M}_k$ della stessa dimensione, e confronta:

$$\text{Var}(\nabla_{\Theta} \mathcal{L}(\Theta; \mathcal{M}_k))$$

in funzione della dimensione del batch. Discuti come l'aumento di N_{batch} influisca sulla varianza e sulla stabilità della convergenza.

Esercizio (Pianificazione del learning rate)

Implementa SGD sullo stesso dataset con tre strategie di learning rate: costante $\eta_k = \eta_0$, decadimento esponenziale $\eta_k = \eta_0 e^{-\gamma k}$, e scaling inverso $\eta_k = \frac{\eta_0}{1 + \gamma k}$. Confronta i tassi di convergenza e discuti quale scelta bilanci meglio velocità e stabilità.

Attenzione

SGD presenta alcuni svantaggi rispetto a GD. In particolare, non c'è modo di verificare se ha raggiunto la convergenza (dato che non possiamo ovviamente calcolare il gradiente di $\mathcal{L}(\theta; X, Y)$ per verificarne la distanza da zero, come richiesto dal primo criterio di arresto) e non possiamo usare l'algoritmo di backtracking, per lo stesso motivo. Di conseguenza, l'algoritmo si fermerà SOLO dopo aver raggiunto il numero fisso di epoche, e dobbiamo impostare manualmente un buon valore per il passo α_k . Questi problemi sono parzialmente risolti da algoritmi più recenti come SGD con Momentum, Adam, AdaGrad, ...

8 Un esempio completo: regressione multi-lineare con GD e SGD

Per concludere la nostra discussione sulla Discesa del Gradiente e sulla sua variante stocastica, consideriamo ora un **esempio completo** in cui implementeremo entrambi gli algoritmi *da zero* (cioè usando solo **numpy**) per risolvere un problema di **regressione multi-lineare** su un dataset reale.

Questo esercizio aiuterà a capire come sia **Gradient Descent (GD)** che **Stochastic Gradient Descent (SGD)** si comportino in uno scenario di training reale, e come scelte diverse (ad es. dimensione del batch, learning rate) influenzino la convergenza.

8.1 Il dataset

Useremo il dataset “**House Prices – Advanced Regression Techniques**” da Kaggle: <https://www.kaggle.com/competitions/house-prices-advanced-regression-techniques>

Il dataset contiene diverse feature che descrivono le case (ad es. numero di stanze, superficie, anno di costruzione) e il **prezzo di vendita** come variabile target.

Per semplicità, useremo solo alcune variabili continue, ad esempio:

- **LotArea** (l'area del lotto in piedi quadrati),
- **OverallQual** (una valutazione complessiva della qualità),
- **YearBuilt** (l'anno di costruzione).

Il nostro obiettivo è predire il prezzo della casa y usando un modello lineare:

$$f_{\Theta}(x) = \Theta^T x,$$

dove $x \in \mathbb{R}^d$ è il vettore delle feature (incluso un termine di bias), e $\Theta = [\Theta_1, \Theta_2, \Theta_3] \in \mathbb{R}^d$ sono i parametri del modello.

8.2 1. Definizione del problema

Definiamo la funzione di perdita **Errore Quadratico Medio (MSE)** come:

$$\mathcal{L}(\Theta; X, Y) = \frac{1}{N} \sum_{i=1}^N (f_{\Theta}(x^{(i)}) - y^{(i)})^2 = \frac{1}{N} \|X\Theta - Y\|_2^2,$$

dove X è la matrice dei dati di forma (N, d) e Y il vettore dei target. Il gradiente della loss rispetto a Θ è:

$$\nabla_{\Theta} \mathcal{L}(\Theta; X, Y) = \frac{2}{N} X^T (X\Theta - Y).$$

8.3 2. Pre-processing dei dati

Il primo passo consiste nel:

- Caricare il dataset (si può scaricare il file **.csv** da Kaggle),
- Selezionare solo le colonne numeriche di interesse,
- Normalizzarle in modo che abbiano media 0 e varianza 1,
- Aggiungere una **colonna di bias** di soli 1 a X .

Prima di entrare nei dettagli implementativi di questi passi, discutiamo la motivazione degli ultimi due.

8.3.1 Normalizzazione dei dati

Prima di applicare un qualsiasi ottimizzatore basato sul gradiente (come Gradient Descent, Stochastic Gradient Descent, o qualsiasi variante), è essenziale **normalizzare o standardizzare i dati**, perché:

1. **Scale delle feature bilanciate:** feature misurate in unità diverse (ad es. metri quadri, anni, valutazioni) producono gradienti di magnitudine molto diversa, rendendo il paesaggio della loss fortemente anisotropo e mal condizionato (come già visto nella sezione precedente).
2. **Convergenza più rapida e regolare:** centrare e scalare le feature rende i contorni della loss più sferici, aiutando i metodi basati sul gradiente a muoversi più direttamente verso il minimo.

Per questo motivo, standardizziamo sempre **sia le feature che il target** nei problemi di regressione:

$$x_j \leftarrow \frac{x_j - \mu_j}{\sigma_j}, \quad y \leftarrow \frac{y - \mu_y}{\sigma_y}.$$

Dopo il training, le predizioni possono essere **denormalizzate** riportandole alla scala originale.

8.3.2 La colonna di bias

Per semplificare l'implementazione del metodo SGD e rendere l'algoritmo risultante indipendente dalla dimensionalità d dello spazio di ingresso, consideriamo la **formulazione matriciale** del problema di ottimizzazione, dove il modello $f_{\Theta}(x^{(i)})$ si scrive:

$$f_{\Theta}(x^{(i)}) = \Theta^T x^{(i)}.$$

Si noti che, sebbene questa formulazione porti a un'implementazione più flessibile, il modello risultante perde l'accesso al termine di **bias** (a volte chiamato **quota**) rispetto alla formulazione scalare:

$$f_{\Theta}(x^{(i)}) = \sum_{j=1}^d W_j x_j^{(i)} + b_i.$$

Per recuperare quel termine, una soluzione tipica è aggiungere una colonna fittizia artificiale a X , composta da tutti 1, in modo che:

$$\tilde{X} = [1, X],$$

e:

$$f_{\Theta}(x^{(i)}) = \Theta^T \tilde{x}^{(i)} = \Theta^T [1, x^{(i)}] = \Theta_1 + \Theta_{2:d+1}^T x^{(i)},$$

recuperando effettivamente il termine di bias.

Possiamo ora passare all'implementazione del pre-processing dei dati per il nostro problema di esempio.

```
import numpy as np
import pandas as pd

# Carica il dataset (scaricato prima da Kaggle)
df = pd.read_csv("data/train.csv")

# Seleziona alcune feature continue utili
features = ["LotArea", "OverallQual", "YearBuilt"]
X = df[features].values
Y = df["SalePrice"].values.reshape(-1, 1)

# Normalizza le feature
X = (X - X.mean(axis=0)) / X.std(axis=0)
Y = (Y - Y.mean(axis=0)) / Y.std(axis=0)

# Aggiunge il termine di bias
X = np.hstack([np.ones((X.shape[0], 1)), X])
N, d = X.shape
print(f"Dimensione dataset: {N} campioni, {d} feature (incluso il bias)")
```

```
Dimensione dataset: 1460 campioni, 4 feature (incluso il bias)
```

8.4 3. Implementazione della Discesa del Gradiente

Implementiamo ora la Discesa del Gradiente usando la formula

$$\Theta^{(k+1)} = \Theta^{(k)} - \eta_k \nabla_{\Theta} \mathcal{L}(\Theta^{(k)}; X, Y).$$

```
def l(Theta, X, Y):
    N = len(Y)
    residuals = X @ Theta - Y
    return (residuals**2).mean()

def grad_l(Theta, X, Y):
    N = len(Y)
```

```

    return (2/N) * X.T @ (X @ Theta - Y)

def gradient_descent(l, grad_l, Theta0, X, Y, lr=1e-3, epochs=500):
    Theta = Theta0
    losses = []
    for k in range(epochs):
        grad = grad_l(Theta, X, Y)
        Theta -= lr * grad
        losses.append(l(Theta, X, Y))
    return Theta, losses

```

8.5 4. Implementazione della Discesa del Gradiente Stocastica

Per SGD, ad ogni iterazione approssimiamo il gradiente completo campionando un **mini-batch** casuale $\mathcal{M}_k$ di dimensione N_{batch} :

$$\nabla_{\Theta} \mathcal{L}(\Theta; \mathcal{M}_k) = \frac{2}{N_{\text{batch}}} X_{\mathcal{M}_k}^T (X_{\mathcal{M}_k} \Theta - Y_{\mathcal{M}_k}).$$

```

def sgd(l, grad_l, Theta0, X, Y, batch_size=64, lr=1e-3, epochs=500):
    Theta = Theta0
    N = len(Y)
    losses = []
    for epoch in range(epochs):
        indices = np.random.permutation(N)
        for start in range(0, N, batch_size):
            batch_idx = indices[start:start+batch_size]
            Xb, Yb = X[batch_idx], Y[batch_idx]
            grad = grad_l(Theta, Xb, Yb)
            Theta -= lr * grad
        losses.append(l(Theta, X, Y))
    return Theta, losses

```

8.6 5. Training e confronto

Possiamo ora allenare entrambi gli algoritmi e confrontarne la convergenza.

```

# Esegue sia GD che SGD
Theta_gd, loss_gd = gradient_descent(l, grad_l, np.zeros((X.shape[1], 1)), X, Y, lr=1e-3, epochs=500)
Theta_sgd, loss_sgd = sgd(l, grad_l, np.zeros((X.shape[1], 1)), X, Y, batch_size=64, lr=1e-3, epochs=500)

import matplotlib.pyplot as plt
plt.plot(loss_gd, label="GD")
plt.plot(loss_sgd, label="SGD")
plt.xlabel("Epoche")
plt.ylabel("Loss")
plt.legend()
plt.title("Convergenza di GD vs. SGD")
plt.grid(alpha=0.3)
plt.show()

```

Nota

Figura del testo originale (non riprodotta qui): un grafico che confronta l'andamento della loss lungo le epoche per GD (curva liscia e regolare) e SGD (curva rumorosa ma che scende più rapidamente all'inizio), entrambe convergenti verso valori simili.

8.7 6. Discussione

- La **Discesa del Gradiente (GD)** calcola il gradiente esatto ad ogni iterazione, garantendo una convergenza regolare, ma è computazionalmente costosa per dataset grandi.
- **SGD**, usando mini-batch, introduce rumore stocastico negli aggiornamenti ma riduce drasticamente il costo computazionale per iterazione. Questo rumore può persino **aiutare** a evitare di rimanere bloccati in minimi locali (per problemi non convessi) e migliora la generalizzazione.
- Il **learning rate** e la **dimensione del batch** controllano congiuntamente il compromesso tra velocità di convergenza e stabilità.

Conviene sperimentare con:

- Diversi valori di **lr** (ad es. $1e-2$, $5e-4$),
- Diverse dimensioni di batch ($N_{\text{batch}} = 1, 32, 128, N$),

8.8 7. Domande per la riflessione

1. Confronta le curve di convergenza di GD e SGD. Quale delle due è più regolare? Quale converge più rapidamente in termini di tempo per epoca?
2. Come influisce la dimensione del batch sul punto finale di convergenza e sulle oscillazioni?
3. Se si sostituisce il learning rate fisso con uno schema decrescente (ad es. $\eta_k = \frac{\eta_0}{1+k}$), come cambia il comportamento?
4. Calcola la norma del gradiente finale $\|\nabla_{\Theta}\mathcal{L}(\Theta^{(E)}; X, Y)\|$ per entrambi gli algoritmi e interpretane il valore.
5. Traccia il valore di $\|\nabla_{\Theta}\mathcal{L}(\Theta^{(k)}; X, Y)\|$ lungo le iterazioni per i due metodi e discuti il comportamento.

Questo esercizio consolida tutti i concetti discussi finora:

- scomposizione della loss come somma sui campioni,
- calcolo esplicito dei gradienti,
- aggiornamenti iterativi tramite Gradient Descent e SGD,
- e i compromessi tra accuratezza, stabilità e costo computazionale.

9 Adam: Adaptive Moment Estimation

Adam è un algoritmo di ottimizzazione che migliora la SGD standard usando sia il **momentum** (per accelerare la convergenza) sia **learning rate adattivi** (per gestire parametri con scale diverse).

Dato un gradiente stocastico all'iterazione k ,

$$g^{(k)} = \nabla_{\Theta}\mathcal{L}(\Theta^{(k)}; \mathcal{M}_k),$$

Adam mantiene due **medie mobili esponenziali (EMA)**:

$$\begin{aligned} m^{(k)} &= \beta_1 m^{(k-1)} + (1 - \beta_1) g^{(k)} && \text{(primo momento: media dei gradienti)} \\ v^{(k)} &= \beta_2 v^{(k-1)} + (1 - \beta_2) (g^{(k)})^2 && \text{(secondo momento: varianza dei gradienti)} \end{aligned}$$

Dopo aver corretto il bias di inizializzazione:

$$\hat{m}^{(k)} = \frac{m^{(k)}}{1 - \beta_1^k}, \quad \hat{v}^{(k)} = \frac{v^{(k)}}{1 - \beta_2^k},$$

l'aggiornamento dei parametri è:

$$\Theta^{(k+1)} = \Theta^{(k)} - \eta \frac{\hat{m}^{(k)}}{\sqrt{\hat{v}^{(k)} + \varepsilon}} g^{(k)}.$$

Scelte tipiche: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$.

9.1 Implementare Adam sul dataset House Prices

Riutilizziamo il dataset Kaggle *House Prices* e risolviamo lo stesso problema di ottimizzazione di prima con Adam.

```
import numpy as np
import pandas as pd

# Carica il dataset
df = pd.read_csv("data/train.csv")

# Seleziona le feature
features = ["LotArea", "OverallQual", "YearBuilt"]
X = df[features].values
Y = df["SalePrice"].values.reshape(-1, 1)

# Normalizza i dati
X_mean, X_std = X.mean(axis=0), X.std(axis=0)
Y_mean, Y_std = Y.mean(), Y.std()

X = (X - X_mean) / X_std
Y = (Y - Y_mean) / Y_std

# Aggiunge la colonna di bias
X = np.hstack([np.ones((X.shape[0], 1)), X])
N, d = X.shape

def l(Theta, X, Y):
    residuals = X @ Theta - Y
    return (residuals**2).mean()

def grad_l(Theta, X, Y):
    N = len(Y)
    return (2/N) * X.T @ (X @ Theta - Y)
```

9.2 Ottimizzatore Adam (con mini-batch)

```
def adam(l,
        grad_l,
        Theta0,
        X, Y,
        beta1=0.9, beta2=0.999, eps=1e-8,
        batch_size=64,
        lr=1e-3,
        epochs=500):
    N, d = X.shape
    Theta = Theta0.copy()

    m = np.zeros_like(Theta)
    v = np.zeros_like(Theta)
    t = 0
    losses = []

    for epoch in range(epochs):
        idx = np.random.permutation(N)
```

```

for start in range(0, N, batch_size):
    batch_idx = idx[start:start+batch_size]
    Xb, Yb = X[batch_idx], Y[batch_idx]
    g = grad_l(Theta, Xb, Yb)

    # Aggiornamenti Adam
    t += 1
    m = beta1 * m + (1 - beta1) * g
    v = beta2 * v + (1 - beta2) * (g * g)
    m_hat = m / (1 - beta1**t)
    v_hat = v / (1 - beta2**t)
    Theta -= lr * (m_hat / (np.sqrt(v_hat) + eps))

losses.append(l(Theta, X, Y))

return Theta, losses

```

9.3 Confronto: SGD vs. Adam (con dati standardizzati)

```

# Allena entrambi
Theta_sgd, loss_sgd = sgd(l, grad_l, np.zeros((X.shape[1], 1)), X, Y, batch_size=64, lr=1e-3,
    epochs=100)
Theta_adam, loss_adam = adam(l, grad_l, np.zeros((X.shape[1], 1)), X, Y, lr=1e-3, batch_size=64,
    epochs=100)

import matplotlib.pyplot as plt
plt.plot(loss_sgd, label="SGD")
plt.plot(loss_adam, label="Adam")
plt.xlabel("Epoca")
plt.ylabel("Loss MSE normalizzata")
plt.title("SGD vs. Adam su dati standardizzati")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

```

Nota

Figura del testo originale (non riprodotta qui): un grafico che confronta la loss MSE normalizzata lungo le epoche per SGD e Adam, con Adam che tipicamente mostra una discesa iniziale più rapida.

9.4 Discussione

Dopo la standardizzazione:

- Sia **SGD** che **Adam** convergono in modo regolare verso minimi quasi identici.
- **Adam** mostra solitamente una riduzione iniziale della loss più rapida, grazie allo scaling adattivo dei gradienti.
- Senza normalizzazione, le magnitudini dei gradienti differiscono tra i parametri, causando l'esplosione o l'annullamento dei learning rate per-parametro di Adam.
- Nello spazio normalizzato, la curvatura della loss è più uniforme, permettendo a metodi adattivi come Adam di funzionare come previsto.

Infine, quando si producono predizioni nella scala originale, ricordarsi di **denormalizzare** l'output:

$$\hat{y}_{\text{reale}} = \hat{y}_{\text{normalizzato}} \cdot \sigma_Y + \mu_Y.$$

*Traduzione italiana del materiale del corso “Statistical and Mathematical Methods for Machine Learning”
(pagina originale in inglese).
Materiale originale di Davide Evangelista — © 2023.*